PHASE 5

Application Factory and Configuration

Core source code: app/main.py and app/config.py

COMICCRAFT • AI COMIC STORY CREATOR • TECHNICAL DOCUMENTATION

1. Core Application Source Code

The sections below reproduce the primary application modules in full, for reference and review.

1.1 Application Factory — app/main.py

Builds the FastAPI application, mounts static files, registers the router, and installs global exception handlers that return a friendly HTML error page for browsers and a uniform JSON envelope for API clients.

Python — app/main.py

```
"""FastAPI application factory and global error handling for ComicCraft."""

from __future__ import annotations

import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import HTMLResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.config import settings
from app.routes import router
from app.templating import templates
from app.utils.helpers import configure_logging

configure_logging()
logger = logging.getLogger("comiccraft")


@asynccontextmanager
async def lifespan(application: FastAPI):
    """Log configuration state at boot so misconfiguration is obvious."""
    logger.info("%s starting in %s mode", settings.APP_NAME, settings.APP_ENV)
    if not settings.gemini_configured:
        logger.warning(
            "GEMINI_API_KEY is missing - comic generation will return a configuration error."
        )
    logger.info("Image provider: %s (%s)", settings.IMAGE_PROVIDER, settings.IMAGE_MODEL)
    yield
    logger.info("%s shutting down", settings.APP_NAME)


def create_app() -> FastAPI:
    """Build and configure the FastAPI application."""
    application = FastAPI(
        title="ComicCraft API",
        description=(
            "ComicCraft turns a story idea into a complete AI-generated comic: "
            "structured panel outline, narration and dialogue, panel illustrations "
            "and a downloadable PDF."
        ),
        version="1.0.0",
```

```
        debug=settings.DEBUG,
        lifespan=lifespan,
    )

    static_dir = str(settings.STATIC_DIR)
    application.mount("/static", StaticFiles(directory=static_dir), name="static")
    application.include_router(router)

    @application.exception_handler(StarletteHTTPException)
    async def http_exception_handler(
        request: Request, exc: StarletteHTTPException
    ):
        """Friendly error page for browsers, JSON for API clients."""
        api_paths = ("/generate-comic", "/test-image", "/health")
        if request.url.path.startswith(api_paths):
            return JSONResponse(
                status_code=exc.status_code,
                content={
                    "success": False,
                    "error": str(exc.detail),
                    "code": "http_error",
                },
            )
        return templates.TemplateResponse(
            request,
            "error.html",
            {
                "request": request,
                "app_name": settings.APP_NAME,
                "status_code": exc.status_code,
                "message": str(exc.detail),
            },
            status_code=exc.status_code,
        )

    @application.exception_handler(RequestValidationError)
    async def validation_exception_handler(
        request: Request, exc: RequestValidationError
    ):
        """Return a compact, safe validation error payload."""
        fields = []
        for error in exc.errors():
            loc = error.get("loc", [])
            location = ".".join(str(p) for p in loc if p != "body")
            fields.append({
```

```
                "field": location or "payload",
                "message": error.get("msg", "Invalid value"),
            })
        return JSONResponse(
            status_code=422,
            content={
                "success": False,
                "error": "Please check your input.",
                "code": "validation_error",
                "fields": fields,
            },
        )

    @application.exception_handler(Exception)
    async def unhandled_exception_handler(request: Request, exc: Exception):
        """Log the technical detail server-side and show a safe message."""
        logger.exception("Unhandled error on %s", request.url.path)
        if request.url.path.startswith(("/generate-comic", "/test-image", "/health")):
            return JSONResponse(
                status_code=500,
                content={
                    "success": False,
                    "error": "Unexpected server error.",
                    "code": "server_error",
                },
            )
        return templates.TemplateResponse(
            request,
            "error.html",
            {
                "request": request,
                "app_name": settings.APP_NAME,
                "status_code": 500,
                "message": "Something went wrong. Please try again.",
            },
            status_code=500,
        )

    return application


app = create_app()
```

1.2 Configuration — app/config.py

A single Settings object, resolved once and cached, that reads every secret, model id, path and generation limit from the environment. No model name or API key is hard-coded anywhere in the application.

Python — app/config.py (abridged)

```
"""Central configuration for ComicCraft."""

from __future__ import annotations

import os
from functools import lru_cache
from pathlib import Path

from dotenv import load_dotenv

BASE_DIR: Path = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")


class Settings:
    """Application settings resolved once at import time."""

    def __init__(self) -> None:
        # --- Application ------------------------------------------------
        self.APP_NAME: str = os.getenv("APP_NAME", "ComicCraft")
        self.APP_ENV: str = os.getenv("APP_ENV", "development")
        self.DEBUG: bool = _as_bool(os.getenv("DEBUG"), True)

        # --- Paths ------------------------------------------------------
        self.BASE_DIR: Path = BASE_DIR
        self.TEMPLATES_DIR: Path = BASE_DIR / "templates"
        self.STATIC_DIR: Path = BASE_DIR / "static"
        self.PANELS_DIR: Path = self.STATIC_DIR / "panels"
        self.EXPORTS_DIR: Path = self.STATIC_DIR / "exports"

        # --- Gemini -----------------------------------------------------
        self.GEMINI_API_KEY: str = os.getenv("GEMINI_API_KEY", "").strip()
        self.GEMINI_OUTLINE_MODEL: str = os.getenv(
            "GEMINI_OUTLINE_MODEL", "models/gemini-1.5-flash"
        ).strip()
        self.GEMINI_STORY_MODEL: str = os.getenv(
            "GEMINI_STORY_MODEL", "models/gemini-1.5-pro"
        ).strip()

        # --- Image generation -------------------------------------------
        # "stable_diffusion" (local) | "huggingface_api" | "gemini_image" | "placeholder"
        self.IMAGE_PROVIDER: str = os.getenv(
            "IMAGE_PROVIDER", "stable_diffusion"
        ).strip().lower()
        self.IMAGE_MODEL: str = os.getenv(
```

```
            "IMAGE_MODEL", "runwayml/stable-diffusion-v1-5"
        ).strip()
        self.HF_API_KEY: str = os.getenv("HF_API_KEY", "").strip()

        # --- Generation rules ------------------------------------------------
        self.PANEL_COUNT: int = _as_int(os.getenv("PANEL_COUNT"), 5)
        self.MAX_PROMPT_LENGTH: int = _as_int(os.getenv("MAX_PROMPT_LENGTH"), 1000)

        self._ensure_directories()

    def _ensure_directories(self) -> None:
        """Create runtime directories that must exist before serving."""
        for directory in (self.STATIC_DIR, self.PANELS_DIR, self.EXPORTS_DIR):
            directory.mkdir(parents=True, exist_ok=True)

    @property
    def gemini_configured(self) -> bool:
        """True when a Gemini API key is present."""
        return bool(self.GEMINI_API_KEY)

    @property
    def image_configured(self) -> bool:
        """True when the selected image provider has what it needs."""
        if self.IMAGE_PROVIDER == "huggingface_api":
            return bool(self.HF_API_KEY)
        if self.IMAGE_PROVIDER == "gemini_image":
            return bool(self.GEMINI_API_KEY)
        return True


@lru_cache(maxsize=1)
def get_settings() -> Settings:
    """Return the cached settings singleton."""
    return Settings()


settings = get_settings()
```